

Optimization Review

SGD, Momentum, Adam, Loss Landscape, Gradient Interference ও Gradient Conflict

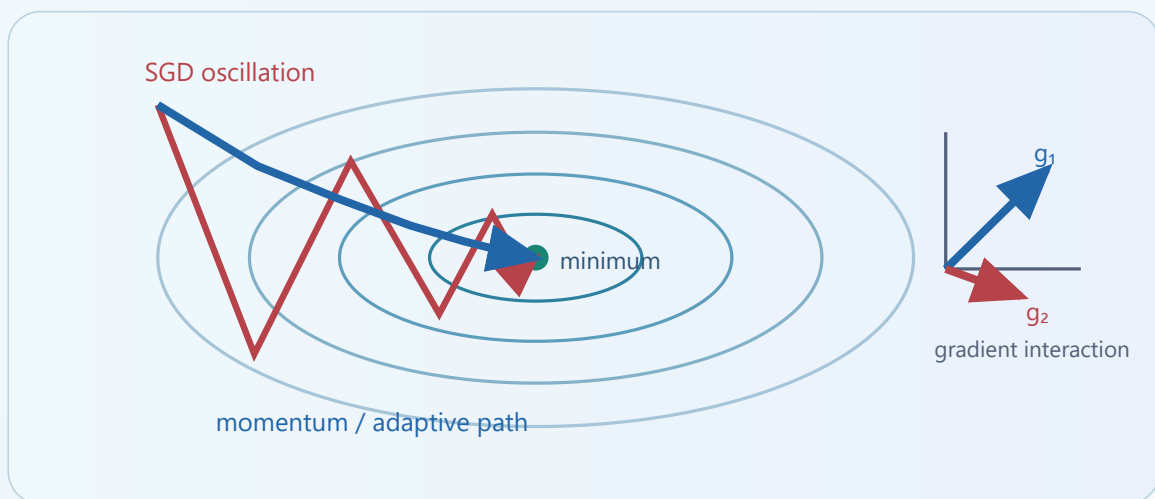
Textbook-style Notes

বাংলা ব্যাখ্যা

Equations

Worked Examples

Practice Problems



Chapter purpose: Optimization algorithm কীভাবে parameter update করে এবং একাধিক task/mini-batch-এর gradient কীভাবে একে অপরকে সাহায্য বা বাধা দেয়—এই দুই ধারণাকে একটি coherent framework-এ বোঝা।

সূচিপত্র

- 01 Optimization-এর ভিত্তি ও notation
- 02 Stochastic Gradient Descent (SGD)
- 03 Momentum ও Nesterov Momentum
- 04 Adam: adaptive first- and second-moment optimization
- 05 SGD, Momentum ও Adam—তুলনা ও ব্যবহারবিধি
- 06 Loss Landscape, curvature, saddle point ও sharpness
- 07 Gradient Interference ও Gradient Conflict
- 08 Conflict measurement ও diagnostic tools
- 09 Conflict mitigation: projection, weighting, replay ও regularization
- 10 Worked examples ও practical workflow
- 11 Summary, cheat sheet, questions ও solutions

Recommended study order

প্রথমে update equations বুঝুন, তারপর landscape দিয়ে optimizer-এর আচরণ visualize করুন, এবং শেষে gradient interaction-কে continual/multi-task learning-এর সাথে যুক্ত করুন। Formula মুখস্থ করার আগে প্রতিটি term-এর ভূমিকা ব্যাখ্যা করতে পারা জরুরি।

Optimization-এর ভিত্তি

Neural network training মূলত একটি optimization problem। Model-এর parameter θ এমনভাবে পরিবর্তন করতে হয় যেন data-এর উপর নির্ধারিত objective বা loss কমে। Continual learning-এ এই কাজটি আরও কঠিন, কারণ নতুন task-এর loss কমাতে গিয়ে পুরোনো task-এর performance নষ্ট হতে পারে।

1.1 Objective function

Training set $D = \{(x_i, y_i)\}$, model f_θ , এবং per-example loss ℓ হলে empirical risk:

$$L(\theta) = (1/N) \sum_{i=1}^N \ell(f_\theta(x_i), y_i) \quad (1)$$

Optimization-এর লক্ষ্য $\theta^* = \arg \min_{\theta} L(\theta)$ । কিন্তু deep network-এর ক্ষেত্রে $L(\theta)$ সাধারণত non-convex, high-dimensional, এবং অসংখ্য saddle point, flat direction ও equivalent solution-এ পূর্ণ। তাই closed-form solution নয়; iterative gradient-based method ব্যবহার করা হয়।

1.2 Gradient কী বলে?

Gradient

$g = \nabla_{\theta} L(\theta)$ হলো parameter space-এ loss সবচেয়ে দ্রুত বাড়ে এমন direction। তাই $-g$ direction-এ ছোট step নিলে loss সাধারণত কমে।

First-order Taylor approximation:

$$L(\theta + \Delta\theta) \approx L(\theta) + \nabla L(\theta)^T \Delta\theta \quad (2)$$

যদি $\Delta\theta = -\eta g$, তবে $L(\theta + \Delta\theta) \approx L(\theta) - \eta \|g\|^2$ । যথেষ্ট ছোট positive learning rate η -এর জন্য loss কমানোর সম্ভাবনা থাকে। তবে step বড় হলে Taylor approximation ভেঙে যায় এবং loss বাড়তেও পারে।

1.3 Batch, mini-batch ও stochastic gradient

পদ্ধতি	Gradient source	সুবিধা	অসুবিধা
Full-batch GD	সম্পূর্ণ training set	Stable, exact empirical gradient	প্রতি update ব্যয়বহুল; large data-তে ধীর
Stochastic GD	একটি sample	খুব দ্রুত update; noise exploration-এ সাহায্য করে	High variance; trajectory অস্থির
Mini-batch SGD	ছোট batch	GPU efficient; speed-variance balance	Batch size ও learning rate tuning দরকার

1.4 Update-এর তিনটি অংশ

1. **Direction:** কোন দিকে parameter সরবে—raw gradient, momentum-smoothed gradient, নাকি adaptive rescaled gradient?
2. **Step size:** কত দূর যাবে—global learning rate এবং parameter-wise scaling দ্বারা নির্ধারিত।
3. **Noise/regularization:** mini-batch noise, weight decay, gradient clipping ইত্যাদি update-কে কীভাবে পরিবর্তন করে?

Core idea

Optimizer শুধু “loss কমায়” না; parameter space-এ কোন পথ দিয়ে, কত noise নিয়ে, কোন curvature direction-এ কী step নেয়—এসবের মাধ্যমে final solution-এর generalization ও forgetting behavior-ও প্রভাবিত করে।

Stochastic Gradient Descent (SGD)

2.1 মূল update rule

Mini-batch B_t -এর gradient $g_t = \nabla_{\theta} L_{B_t}(\theta_t)$ হলে:

$$\theta_{t+1} = \theta_t - \eta g_t \quad (3)$$

এখানে η learning rate নামের "stochastic" অংশটি আসে কারণ mini-batch gradient full-data gradient-এর একটি noisy estimate:

$$g_t = \nabla L(\theta_t) + \xi_t, \text{ যেখানে } E[\xi_t] \approx 0 \quad (4)$$

2.2 Intuition: পাহাড় থেকে নামা

Loss surface-কে পাহাড়ি terrain হিসেবে ভাবুন। SGD প্রতিটি mini-batch দেখে স্থানীয় ঢাল অনুমান করে এবং নিচের দিকে এক ধাপ যায়। Mini-batch বদলালে ঢালের estimate-ও বদলায়, তাই পথটি zigzag হয়। এই noise সবসময় খারাপ নয়—shallow basin বা saddle region থেকে বের হতে সাহায্য করতে পারে এবং তুলনামূলক flat solution-এর দিকে bias তৈরি করতে পারে।

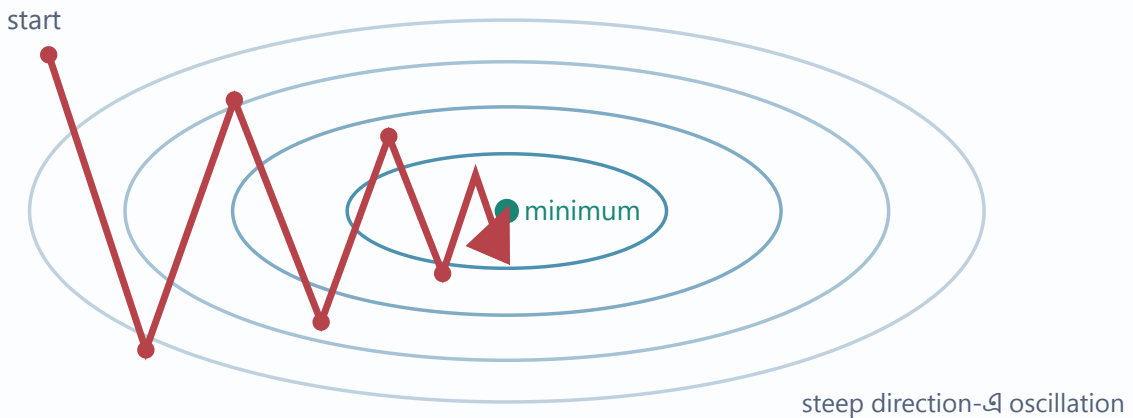


Figure 2.1 — Ill-conditioned valley-তে SGD steep direction বরাবর oscillate করতে পারে।

2.3 Learning rate-এর প্রভাব

Learning rate	Observed behavior	Typical symptom
খুব ছোট	Safe কিন্তু ধীর progress	Loss কমছে, কিন্তু training budget শেষ হয়ে যাচ্ছে
উপযুক্ত	Fast stable descent	Training loss নিয়মিত কমে, validation উন্নত হয়
খুব বড়	Minimum overshoot বা divergence	Loss oscillate করে, NaN/Inf হতে পারে

Learning-rate schedule

- **Step decay:** নির্দিষ্ট epoch পর η কমানো।
- **Exponential decay:** $\eta_t = \eta_0 \gamma^t$
- **Cosine annealing:** শুরুতে বড়, শেষে smoothভাবে ছোট learning rate।
- **Warm-up:** training-এর প্রথম দিকে ছোট η থেকে ধীরে বাড়ানো; large batch/Transformer-এ উপকারী।

2.4 Weight decay ও SGD

L2 penalty যোগ করলে objective হয় $L(\theta) + (\lambda/2)\|\theta\|^2$ । Gradient update:

$$\theta_{t+1} = (1 - \eta\lambda)\theta_t - \eta\nabla L(\theta_t) \quad (5)$$

অর্থাৎ প্রতিটি step-এ weight সামান্য shrink করে। Plain SGD-এর ক্ষেত্রে L2 regularization এবং weight decay ঘনিষ্ঠভাবে equivalent; adaptive optimizer-এ decoupled weight decay (যেমন AdamW) আলাদাভাবে বিবেচনা করা ভালো।

Worked mini-example

ধরা যাক $\theta = [2, -1]$, gradient $g = [0.4, -0.2]$, এবং $\eta = 0.1$ । তাহলে

$$\theta' = [2, -1] - 0.1[0.4, -0.2] = [1.96, -0.98]$$

প্রথম parameter কমেছে এবং দ্বিতীয়টি কম negative হয়েছে—দুটিই negative-gradient direction অনুসরণ করছে।

2.5 SGD-এর শক্তি ও সীমাবদ্ধতা

Strengths

সরল, memory-efficient, behavior সহজে বোঝা যায়; tuned schedule সহ vision task-এ শক্তিশালী generalization দিতে পারে।

Limitations

Ill-conditioned landscape-এ zigzag; সব parameter একই global learning rate পায়; schedule tuning-sensitive।

Momentum

Momentum বর্তমান gradient-এর পাশাপাশি অতীতের consistent direction মনে রাখে। ফলে যে direction-এ gradient বারবার একই sign দেয় সেখানে গতি জমে, আর যে direction-এ gradient sign বদলায় সেখানে oscillation আংশিক cancel হয়।

3.1 Classical momentum update

$$\begin{aligned} v_t &= \beta v_{t-1} + g_t \\ \theta_{t+1} &= \theta_t - \eta v_t \end{aligned} \quad (6)$$

কিছু library equivalent scaling ব্যবহার করে $v_t = \beta v_{t-1} + (1-\beta)g_t$ । Convention আলাদা হলেও intuition একই। β সাধারণত 0.9-এর কাছাকাছি; এটি effective memory length আনুমানিক $1/(1-\beta)$ step দেয়। তাই $\beta=0.9$ প্রায় 10-step history এবং $\beta=0.99$ প্রায় 100-step history smooth করে।

3.2 কেন momentum দ্রুত?

একটি elongated valley বিবেচনা করুন। Horizontal direction-এ minimum-এর দিকে gradient consistent, vertical direction-এ gradient alternating। Momentum horizontal component accumulate করে কিন্তু vertical positive/negative component cancel করে। ফল: কম zigzag, দ্রুত forward progress।

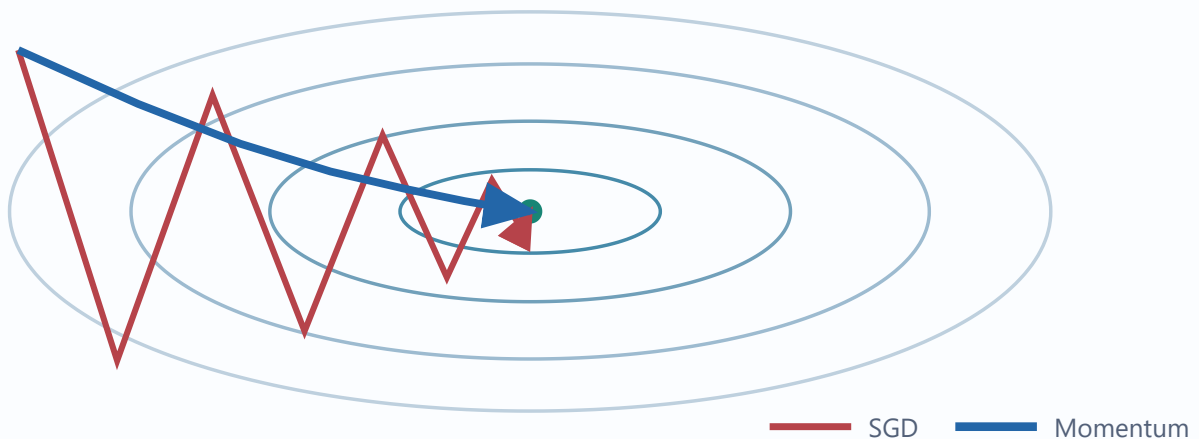


Figure 3.1 — একই valley-তে momentum oscillation কমিয়ে consistent direction-এ দ্রুত এগোয়।

3.3 Exponential moving average view

Momentum buffer expand করলে:

$$v_t = g_t + \beta g_{t-1} + \beta^2 g_{t-2} + \dots \quad (7)$$

অতীত gradient exponentially decaying weight পায়। তাই momentum একধরনের low-pass filter: high-frequency batch noise কমায়, persistent signal ধরে রাখে।

3.4 Nesterov Accelerated Gradient (NAG)

Classical momentum বর্তমান location-এ gradient মাপে। Nesterov আগে momentum দিয়ে model কোথায় যেতে পারে সেই “look-ahead” point অনুমান করে, তারপর সেখানে gradient মাপে:

$$\begin{aligned} g_t &= \nabla L(\theta_t - \eta \beta v_{t-1}) \\ v_t &= \beta v_{t-1} + g_t \quad \theta_{t+1} = \theta_t - \eta v_t \end{aligned} \quad (8)$$

Intuition: গাড়ি সামনে কোথায় যাবে তা দেখে আগেই steering correction করা। Steep পরিবর্তনের কাছাকাছি এটি overshoot কমাতে পারে। Framework implementation-এ algebraic form কিছুটা ভিন্ন দেখা গেলেও look-ahead ধারণাই মূল।

3.5 Momentum-এর practical considerations

- Learning rate ও momentum একে অপরের সাথে যুক্ত। β বাড়ালে accumulated step বড় হতে পারে; learning rate retune করতে হয়।
- Training শুরুর দিকে velocity শূন্য, তাই warm-up behavior থাকে।
- Task boundary-তে পুরোনো momentum buffer নতুন task-এর gradient-এর বিপরীত হতে পারে। Continual learning-এ optimizer state reset করা হবে কি না—এটি গুরুত্বপূর্ণ design choice।
- Gradient clipping প্রয়োগের order documentation অনুযায়ী বুঝতে হবে; সাধারণত gradient clip করার পর optimizer step নেয়।

Exam insight

SGD ও momentum-এর পার্থক্য শুধু “momentum দ্রুত” নয়। Momentum gradient history ব্যবহার করে direction smooth ও accelerate করে; তাই optimization path, oscillation এবং task switch response—তিনটিই বদলায়।

Adam

Adam অর্থ Adaptive Moment Estimation। এটি gradient-এর exponential moving average (first moment) এবং squared gradient-এর exponential moving average (second raw moment) রাখে। ফলস্বরূপ প্রতিটি parameter আলাদা effective step size পায়।

4.1 Adam update equations

$$\begin{aligned} g_t &= \nabla L(\theta_{t-1}) \\ m_t &= \beta_1 m_{t-1} + (1-\beta_1)g_t \\ v_t &= \beta_2 v_{t-1} + (1-\beta_2)g_t^2 \end{aligned} \quad (9)$$

শুরুতে $m_0=0, v_0=0$, তাই estimates zero-এর দিকে biased। Bias correction:

$$\hat{m}_t = m_t / (1-\beta_1^t), \quad \hat{v}_t = v_t / (1-\beta_2^t) \quad (10)$$

Final parameter update:

$$\theta_t = \theta_{t-1} - \eta \cdot \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon) \quad (11)$$

4.2 প্রতিটি term-এর অর্থ

Term	Role	Typical default
m_t	Gradient direction-এর smoothed estimate; momentum-এর মতো	$\beta_1=0.9$
v_t	Gradient magnitude/scale-এর smoothed squared estimate	$\beta_2=0.999$
ϵ	Division-by-zero ও numerical instability এড়ায়	10^{-8} (implementation-dependent)
η	Base learning rate	প্রায় 10^{-3} , task অনুযায়ী tune

4.3 Adaptive scaling intuition

যে parameter-এ gradient বারবার বড়, তার $\hat{v}$ বড় হবে এবং denominator update কমাবে। যে parameter-এ gradient sparse বা ছোট, তার denominator ছোট হওয়ায় relative update বড় হতে পারে। তাই sparse feature, embeddings বা uneven gradient scale-এ Adam দ্রুত progress দিতে পারে।

এক-step numerical intuition

প্রথম step-এ ধরা যাক একটি scalar gradient $g_1=0.2$, $\beta_1=0.9$, $\beta_2=0.999$ ।

$$m_1=0.02, v_1=0.00004$$

$$\hat{m}_1=0.2, \hat{v}_1=0.04$$

তাই normalized direction প্রায় $0.2/\sqrt{0.04} = 1$; update প্রায় $-\eta$ । Bias correction না করলে প্রথম step অস্বাভাবিকভাবে ছোট হতো।

4.4 Adam বনাম AdamW

Adam-এর adaptive scaling-এর ভিতরে L2 penalty gradient যোগ করলে weight shrink parameter-wise preconditioned হয়। AdamW weight decay-কে loss gradient থেকে decouple করে:

$$\theta_t \leftarrow (1-\eta\lambda)\theta_{t-1} - \eta \cdot \hat{m}_t / (\sqrt{\hat{v}_t + \epsilon}) \quad (12)$$

এই distinction গুরুত্বপূর্ণ, কারণ “L2 regularization” এবং “weight decay” adaptive optimizer-এ এক নয়। Modern deep learning recipe-তে AdamW বহুল ব্যবহৃত।

4.5 Adam-এর সুবিধা

- Fast initial convergence এবং তুলনামূলক কম manual scaling।
- Different parameter scale ও sparse gradients ভালোভাবে সামলাতে পারে।
- Transformer, language model এবং দ্রুত prototyping-এ শক্তিশালী baseline।
- Gradient history ও scale দুটোই ব্যবহার করে noisy direction smooth করে।

4.6 সতর্কতা

- Adaptive method training loss দ্রুত কমালেও সব task-এ final generalization সেরা হবে—এমন নিশ্চয়তা নেই।
- Optimizer state-এর জন্য parameter প্রতি অতিরিক্ত m ও v রাখতে হয়; memory cost বেশি।
- Task boundary-তে old moments নতুন gradient-কে distort করতে পারে। Reset, partial reset, বা continuous state—experiment দিয়ে নির্ধারণ করা উচিত।
- খুব বড় learning rate, inappropriate ϵ , বা mixed-precision issue instability তৈরি করতে পারে।

Common misconception

Adam “learning rate-এর প্রয়োজন দূর করে” না। এটি per-parameter adaptive scaling দেয়, কিন্তু base learning rate, schedule, weight decay ও clipping এখনও গুরুত্বপূর্ণ hyperparameter।

SGD, Momentum ও Adam: তুলনা

Dimension	SGD	Momentum	Adam / AdamW
Direction	Current mini-batch gradient	Current + historical gradient	Smoothed gradient, squared-gradient দ্বারা rescaled
Extra state	প্রায় নেই	একটি velocity tensor	দুটি tensors: first ও second moments
Noise handling	Raw/noisy	History দিয়ে smooth	Smooth + coordinate-wise normalization
Ill-conditioning	Zigzag হতে পারে	অনেকটা উন্নত	Adaptive scale দিয়ে প্রায়ই দ্রুত
Initial convergence	তুলনামূলক ধীর	SGD-এর চেয়ে দ্রুত	প্রায়ই দ্রুততম
Memory	সর্বনিম্ন	মাঝারি	সর্বাধিক
Typical use	Simple baseline, memory-limited setup	Vision/classical deep nets	Transformers, sparse gradients, prototyping
Task-boundary issue	State carryover কম	Velocity conflict করতে পারে	First/second moments carryover করতে পারে

5.1 কোন optimizer বেছে নেব?

- 1 Strong default স্থাপন করুন:** Transformer/sparse-gradient task-এ AdamW; many vision settings-এ Momentum SGD ভালো starting point।
- 2 Fair tuning করুন:** একই learning rate দিয়ে তিন optimizer তুলনা করা fair নয়। প্রত্যেকটির উপযুক্ত range ও schedule দরকার।
- 3 শুধু training loss নয়:** validation accuracy, calibration, forgetting, convergence time ও memory cost মাপুন।
- 4 Continual setting-এ boundary test করুন:** optimizer state carry বনাম reset—দুটিই compare করুন।

5.2 PyTorch-style pseudocode

Common training skeleton

```

for x, y in loader:
    optimizer.zero_grad()
    logits = model(x)
    loss = criterion(logits, y)
    loss.backward()
    clip_grad_norm_(model.parameters(), max_norm) # optional
    optimizer.step()

```

Optimizer বদলালেও core loop একই থাকে; পার্থক্যটি `optimizer.step()` -এর ভিতরের state ও update rule-এ। Continual learning regularizer থাকলে `loss = task_loss + \lambda * continual_penalty` করতে হবে। Penalty function argument হিসেবে দিলেই হবে না; total loss-এ সত্যিই যোগ করতে হবে।

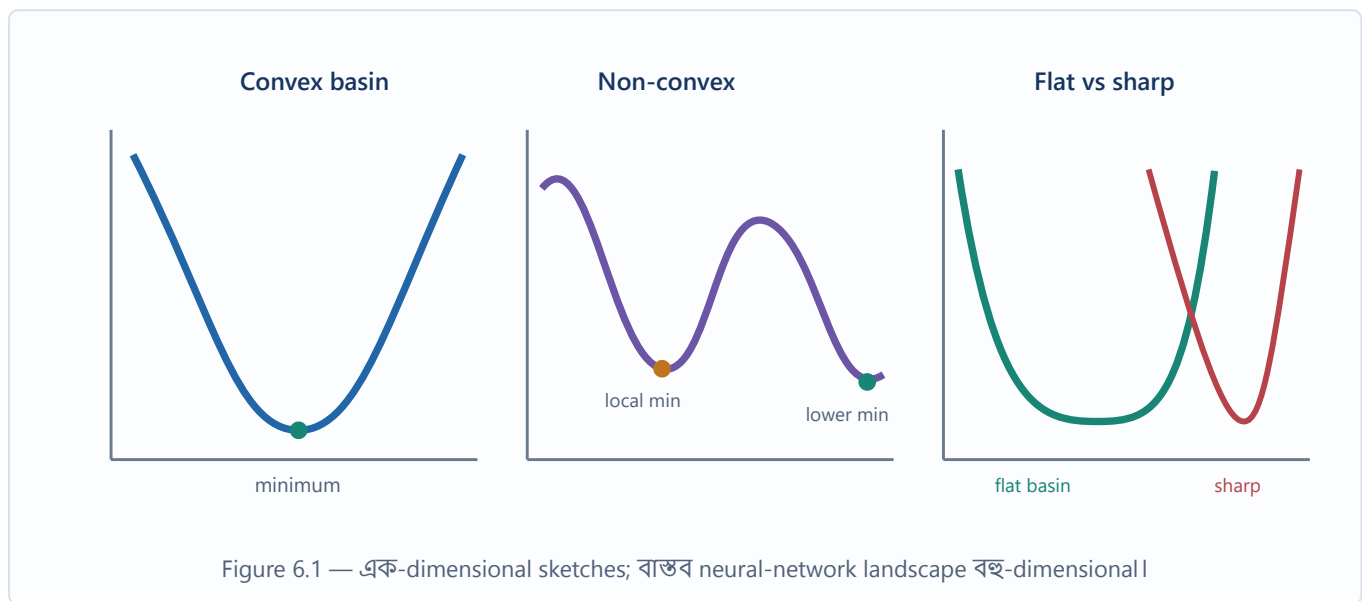
Comparison answer template

একটি ভালো exam answer-এ চারটি জিনিস থাকবে: (1) update equation, (2) historical state, (3) landscape behavior, এবং (4) computational trade-off। শুধু "Adam adaptive, SGD simple" লিখলে explanation অসম্পূর্ণ।

Loss Landscape

Loss landscape হলো parameter vector θ থেকে scalar loss $L(\theta)$ -এ mapping-এর geometric picture। বাস্তবে parameter dimension লাখ-কোটি হতে পারে; visual plot সাধারণত দুই direction-এর slice বা projection মাত্র। তবু curvature, valley, saddle এবং optimization difficulty বোঝার জন্য ধারণাটি অত্যন্ত কার্যকর।

6.1 Landscape-এর প্রধান feature



Local/global minimum

Local minimum-এ নিকটবর্তী ছোট perturbation loss বাড়ায়। **Global minimum** পুরো parameter space-এ সর্বনিম্ন loss দেয়। Deep learning-এ বহু parameter permutation ও overparameterization-এর কারণে অসংখ্য functionally equivalent minimum থাকতে পারে।

Saddle point

Saddle point-এ gradient শূন্য বা খুব ছোট হলেও কিছু direction-এ curvature positive, অন্য direction-এ negative। তাই এটি এক direction-এ minimum, অন্য direction-এ maximum-এর মতো। High-dimensional non-convex optimization-এ saddle behavior গুরুত্বপূর্ণ।

Plateau ও flat region

Gradient খুব ছোট হলে optimizer ধীরে চলে। Momentum persistent ছোট gradient accumulate করে সাহায্য করতে পারে; adaptive scaling কখনও ছোট coordinate gradient amplify করতে পারে। তবে true flatness ও noisy estimate আলাদা করে দেখা দরকার।

6.2 Curvature ও Hessian

Gradient slope দেয়; Hessian দ্বিতীয় derivative matrix curvature দেয়:

$$H(\theta) = \nabla^2_{\theta} L(\theta)$$

(13)

Hessian eigenvalue λ_i একটি principal direction-এর curvature নির্দেশ করে:

- $\lambda_i > 0$: সেই direction-এ bowl-like positive curvature।
- $\lambda_i < 0$: downward curvature; saddle/maximum direction।
- $\lambda_i \approx 0$: flat direction।

6.3 Condition number ও zigzag

Positive curvature region-এ আনুমানিক condition number:

$$\kappa = \lambda_{\max} / \lambda_{\min}$$

(14)

κ বড় হলে এক direction খুব steep এবং আরেকটি খুব flat—এটিই ill-conditioning। একটি global learning rate steep direction-এর জন্য ছোট রাখতে হয়, ফলে flat direction-এ progress ধীর হয়। Momentum oscillation smooth করে; Adam coordinate-wise scale adjust করে। তবে Adam-এর diagonal rescaling Hessian-এর full rotation capture করে না।

6.4 Sharp বনাম flat minima

Sharp minimum-এর আশেপাশে parameter সামান্য বদলালে loss দ্রুত বাড়ে; flat basin-এ perturbation তুলনামূলক সহনীয়। Flatness প্রায়ই robustness/generalization intuition দেয়, কিন্তু parameter reparameterization, scale symmetry এবং measurement method-এর উপর sharpness নির্ভর করতে পারে। তাই “flat মানেই সবসময় better” একটি অতিরিক্ত সরলীকরণ।

6.5 Loss barrier ও task geometry

Task A-এর solution θ_A থেকে Task B-এর solution θ_B -এ যাওয়ার পথে Task A loss বাড়লে একটি loss barrier তৈরি হয়। Continual learning-এ নতুন task update পুরোনো basin থেকে model-কে বের করে দিলে forgetting হয়। Regularization method গুরুত্বপূর্ণ parameter-কে বেশি নড়তে বাধা দেয়; replay পুরোনো task gradient আবার update-এ যুক্ত করে।

Landscape–optimizer connection

SGD local slope অনুসরণ করে, momentum direction history দিয়ে valley traverse করে, Adam gradient scale অনুযায়ী coordinate rescale করে। অর্থাৎ optimizer landscape বদলায় না; landscape-এর উপর চলার metric/path বদলায়।

Gradient Interference ও Gradient Conflict

Multi-task বা continual learning-এ একাধিক objective একই parameter ভাগ করে। একটি update যদি এক task-এর loss কমায় কিন্তু অন্য task-এর loss বাড়ায়, তাকে gradient conflict-এর মাধ্যমে বোঝানো যায়। “Interference” broader outcome; “conflict” তার first-order geometric signal।

7.1 দুই task-এর gradient

Task A এবং B-এর loss যথাক্রমে $L_A(\theta)$ ও $L_B(\theta)$:

$$g_A = \nabla L_A(\theta), \quad g_B = \nabla L_B(\theta) \quad (15)$$

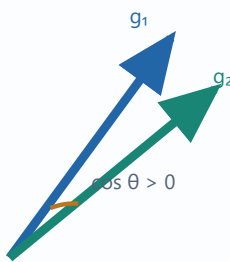
যদি Task B update $\Delta\theta = -\eta g_B$ নেওয়া হয়, Task A loss-এর first-order change:

$$\Delta L_A \approx g_A^T \Delta\theta = -\eta g_A^T g_B \quad (16)$$

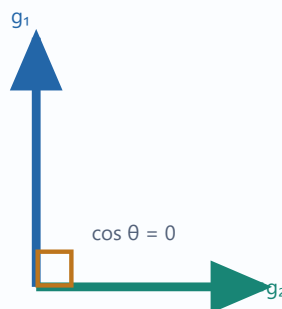
এখানে dot product-এর sign সিদ্ধান্ত দেয়:

Relationship	$g_A \cdot g_B$	Interpretation
Aligned / synergistic	Positive	B-এর negative-gradient step A loss-ও first order-এ কমায়; positive transfer
Orthogonal / neutral	Zero	B step A loss-এ first-order change আনে না; second-order effect থাকতে পারে
Conflicting	Negative	B step A loss বাড়ায়; interference/forgetting-এর ঝুঁকি

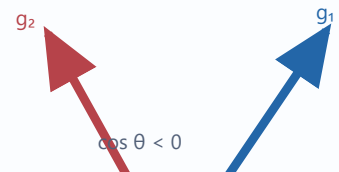
Aligned



Orthogonal



Conflicting



Angle direction relation দেখায়; vector length gradient magnitude দেখায়।

Figure 7.1 — Gradient angle দিয়ে synergy, neutrality ও conflict-এর first-order geometry।

7.2 Gradient interference বনাম conflict

Gradient conflict

নির্দিষ্ট parameter state ও batch/task pair-এ gradient dot product negative হওয়া। এটি local এবং first-order criterion।

Gradient interference

একটি update বা training phase-এর কারণে অন্য sample/task-এর loss বা performance খারাপ হওয়া। এটি observed effect; finite step, curvature, optimizer state ও data order দ্বারা প্রভাবিত।

তাই negative dot product conflict-এর শক্তিশালী signal, কিন্তু সব forgetting ব্যাখ্যা করে না। Dot product zero হলেও second-order term $(1/2)\Delta\theta^T H_A \Delta\theta$ Task A loss বাড়াতে পারে। আবার small negative dot product হলেও tiny learning rate-এ practical damage সামান্য হতে পারে।

7.3 Continual learning-এ conflict কোথা থেকে আসে?

- **Different label rules:** একই representation দুই task ভিন্ন decision boundary চায়।
- **Distribution shift:** input statistics বদলে shared feature update হয়।
- **Limited capacity:** সব task-এর solution একই parameter subspace-এ fit না-ও করতে পারে।
- **Sequential data:** পুরোনো task sample না থাকায় current gradient একপাক্ষিক হয়।
- **Shared head conflict:** class-incremental setup-এ output competition বাড়ে।
- **Optimizer state:** momentum/Adam moments পুরোনো direction বহন করে task switch-এর response বদলায়।

7.4 Stability–plasticity dilemma

Plasticity হলো নতুন task দ্রুত শেখার ক্ষমতা; **stability** হলো পুরোনো knowledge ধরে রাখা। Current-task gradient সম্পূর্ণ অনুসরণ করলে plasticity বেশি কিন্তু forgetting বাড়তে পারে। Parameter শক্তভাবে freeze/regularize করলে stability বেশি কিন্তু নতুন task শেখা বাধাগ্রস্ত হতে পারে। Continual-learning method এই trade-off balance করে।

$$L_{\text{total}}(\theta) = L_{\text{new}}(\theta) + \lambda \Omega(\theta; \theta_{\text{old}}, \text{importance}) \quad (17)$$

SI, MAS, EWC-এর মতো method-এ Ω গুরুত্বপূর্ণ parameter-এর movement penalize করে। Replay-based method পুরোনো data/representation থেকে gradient যোগ করে objective-কে multi-task approximation-এ পরিণত করে।

Conflict মাপা ও diagnose করা

8.1 Dot product

$$s_{\text{dot}} = \mathbf{g}_i^T \mathbf{g}_j \quad (18)$$

Sign conflict বলে, magnitude interaction strength-এর scale দেয়। তবে gradient norm বড় হলে dot product বড় হয়; direction comparison-এর জন্য normalized metric দরকার।

8.2 Cosine similarity

$$\cos(\mathbf{g}_i, \mathbf{g}_j) = (\mathbf{g}_i^T \mathbf{g}_j) / (\|\mathbf{g}_i\| \|\mathbf{g}_j\| + \epsilon) \quad (19)$$

Cosine range	Meaning	Suggested interpretation
Near +1	Strong alignment	Shared update উভয় objective-কে সাহায্য করতে পারে
Near 0	Near orthogonal	First-order interaction কম; norms ও curvature দেখুন
Near -1	Strong opposition	Direct conflict; projection/replay/weighting বিবেচনা করুন

8.3 Conflict rate

অনেক batch/task pair-এ cosine negative হওয়ার fraction:

$$\text{Conflict Rate} = (\# \text{ pairs with } \cos < 0) / (\text{total measured pairs}) \quad (20)$$

Average cosine একা misleading হতে পারে। কিছু বড় positive value অনেক frequent ছোট negative value ঢেকে দিতে পারে। তাই mean, median, negative fraction, quantile ও norm একসাথে report করা ভালো।

8.4 Gradient matrix

K task-এর gradient stack করে $\mathbf{G} = [\mathbf{g}_1, \dots, \mathbf{g}_K]$ নিলে pairwise Gram matrix:

$$\mathbf{C} = \mathbf{G}^T \mathbf{G} \quad (21)$$

C_{ij} dot product। Normalized version cosine matrix। Heatmap-এ block structure দেখা গেলে কিছু task group aligned এবং কিছু group conflicting হতে পারে। এটি task grouping, routing বা separate adapter design-এ সাহায্য করে।

8.5 Layer-wise analysis

Full-model cosine একটি aggregate। Early encoder aligned কিন্তু classifier head conflicting হতে পারে। তাই layer/module-wise gradient cosine মাপলে conflict-এর location বোঝা যায়। Practicalভাবে:

- Shared backbone বনাম task-specific head আলাদা মাপুন।
- Gradient norm খুব ছোট layer-এ cosine noisy হতে পারে; threshold ব্যবহার করুন।
- Batch-level metric বহু step average করুন; এক step থেকে সিদ্ধান্ত নেবেন না।
- Train/eval mode, dropout randomness ও batch normalization statistics consistent রাখুন।

8.6 Functional interference মাপা

Gradient metric-এর পাশাপাশি একটি temporary update নিয়ে অন্য task-এর loss change মাপা যায়:

1. Parameter snapshot নিন।
2. Task B-এর এক update করুন।
3. Task A-এর probe batch loss before/after তুলনা করুন।
4. Parameter restore করুন।

এটি finite-step, optimizer state ও curvature effect capture করে, তবে computationally বেশি ব্যয়বহুল।

Measurement pitfall

Raw flattened gradient তুলনা করার সময় parameter order একই হতে হবে এবং `grad is None` parameter consistentভাবে handle করতে হবে। Accidental mismatch সম্পূর্ণ ভুল cosine তৈরি করতে পারে।

Gradient conflict কমানোর কৌশল

9.1 Loss weighting

$$L(\theta) = \sum_{k=1}^K \alpha_k L_k(\theta) \Rightarrow g = \sum_{k=1}^K \alpha_k g_k \quad (22)$$

Weight α_k task influence নিয়ন্ত্রণ করে। Equal weight সহজ baseline, কিন্তু loss scale ভিন্ন হলে একটি task dominate করতে পারে। Dynamic weighting gradient norm, uncertainty বা training rate balance করতে পারে। Weighting conflict দূর না-ও করতে পারে; combined direction বদলায়।

9.2 Gradient projection / surgery

যদি $g_i \cdot g_j < 0$, g_i থেকে g_j -এর conflicting component বাদ দেওয়া যায়:

$$g'_i = g_i - (g_i^T g_j / \|g_j\|^2) g_j \quad (23)$$

Projection-এর পর $g'_i \cdot g_j = 0$ (ideal arithmetic), অর্থাৎ g_j -এর বিরুদ্ধে first-order component থাকে না। PCGrad-এর মতো method pairwise conflicting gradient project করে। Task order randomization ও multiple projections-এর order ফলকে প্রভাবিত করতে পারে।

Projection example

$g_1=[2,1]$, $g_2=[-1,1]$ । Dot product $-2+1=-1$, তাই conflict। $\|g_2\|^2=2$ ।

$$g'_1 = [2,1] - (-1/2)[-1,1] = [1.5,1.5]$$

Check: $[1.5,1.5] \cdot [-1,1]=0$ । Conflicting component remove হয়েছে।

9.3 Replay

Memory buffer থেকে পুরোনো sample নিয়ে current batch-এর সাথে train করলে:

$$L_{\text{step}} = L_{\text{current}} + \alpha L_{\text{replay}} \quad (24)$$

Replay পুরোনো task-এর gradient সরাসরি update-এ ফিরিয়ে আনে। Buffer selection, class balance, memory size ও privacy/storage constraint গুরুত্বপূর্ণ। Gradient Episodic Memory ধরনের method replay gradient-কে constraint হিসেবেও ব্যবহার করতে পারে।

9.4 Parameter regularization

পুরোনো task-এর জন্য গুরুত্বপূর্ণ parameter θ_i যেন বেশি না বদলায়:

$$\Omega(\theta) = \sum_i I_i (\theta_i - \theta_i^{\text{old}})^2 \quad (25)$$

Method family	Importance intuition	Main effect
EWC	Fisher-based sensitivity	Important weights-এর quadratic movement penalty
SI	Training trajectory-তে parameter contribution	Past loss reduction-এ useful weights protect
MAS	Output-function sensitivity	Output-এ sensitive weights protect; labels ছাড়াও estimate করা যায়

Regularization coefficient খুব ছোট হলে forgetting, খুব বড় হলে intransigence—নতুন task শেখে না। Importance normalize/clamp এবং task count বাড়লে penalty accumulation কীভাবে হবে তা design করতে হয়।

9.5 Architectural separation

- Task-specific heads বা adapters conflict-prone parameter আলাদা করে।
- Routing/gating input অনুযায়ী expert নির্বাচন করে।
- Parameter isolation stability বাড়ায়, কিন্তু model size/task identity requirement বাড়তে পারে।

9.6 Representation-level approaches

Feature distillation, logit matching বা contrastive objective old representation ধরে রাখতে সাহায্য করে। Gradient conflict output loss-এর বাইরে representation drift থেকেও আসতে পারে; তাই feature-space constraint কার্যকর হতে পারে।

9.7 কোন strategy কখন?

Constraint	Promising approach	Reason
Old data রাখা যায়	Replay + suitable optimizer	Past gradient-এর direct approximation
Data রাখা যায় না	SI/MAS/EWC, distillation	Parameter/function summary দিয়ে memory
Task gradients একই step-এ পাওয়া যায়	PCGrad/gradient surgery/weighting	Conflict সরাসরি measure ও modify করা যায়
Model capacity বাড়ানো যায়	Adapters, experts, separate heads	Conflicting subspace isolate করা যায়
Compute খুব সীমিত	Simple regularization বা small replay	Second backward/projection overhead এড়ানো যায়

Worked Examples ও Practical Workflow

10.1 Cosine similarity calculation

Example A — conflict strength

$g_A = [3, -1, 2]$ এবং $g_B = [-2, 1, 0]$ ।

$$g_A \cdot g_B = 3(-2) + (-1)(1) + 2(0) = -7$$

$$\|g_A\| = \sqrt{14}, \quad \|g_B\| = \sqrt{5}, \\ \cos = -7/\sqrt{70} \approx -0.837$$

Strong negative cosine নির্দেশ করে gradients প্রায় বিপরীত। B-এর step A loss first-order-এ বাড়াবে।

10.2 Momentum numerical sequence

Example B — consistent gradient accumulation

Scalar parameter, $\beta=0.9$, $\eta=0.1$, এবং ধারাবাহিক gradient $g_1=g_2=g_3=1$ ।

$$v_1=1; \quad v_2=1.9; \quad v_3=2.71$$

Update magnitudes 0.1, 0.19, 0.271—consistent direction-এ velocity জমছে। কিন্তু gradient sign alternating হলে accumulation cancel হয়ে oscillation কমে।

10.3 Task B training-এর পর forgetting

Task A accuracy before B = 92%, after B = 71%, Task B accuracy = 88% হলে:

$$\text{Forgetting}(A) = 92 - 71 = 21 \text{ percentage points}$$

(26)

“21% forgetting” বলা ambiguous; সঠিক ভাষা “21 percentage points”। Relative drop চাইলে $(92-71)/92 \approx 22.8\%$ । Report-এ metric definition স্পষ্ট রাখুন।

10.4 একটি complete experiment workflow

- 1 **Baseline:** একই model initialization/data order দিয়ে SGD, Momentum, AdamW আলাদাভাবে tune করুন।
- 2 **Metrics:** current accuracy, retained accuracy, average accuracy, forgetting, train time ও optimizer memory লিখুন।

- 3 **Gradient logging:** নির্দিষ্ট interval-এ current ও replay/probe batch gradient cosine এবং norm নিন।
- 4 **Boundary study:** task switch-এর আগে/পরে optimizer state reset বনাম carry তুলনা করুন।
- 5 **Mitigation:** replay/regularization/projection একটি করে যোগ করুন; একইসাথে অনেক change নয়।
- 6 **Ablation:** regularization strength, buffer size, projection on/off, task order ও seeds পরীক্ষা করুন।
- 7 **Interpretation:** শুধু final table নয়; conflict rate ও forgetting-এর relationship plot করুন। Correlation causation নয়—functional probe দিয়ে confirm করুন।

10.5 Recommended logging table

Step/Task	Optimizer	LR	Current Acc.	Old Acc.	Cosine	Grad Norm	Conflict?
T2 / 100	Momentum	0.03	63.2	86.1	-0.41	2.73	Yes
T2 / 200	Momentum	0.03	75.8	80.4	-0.18	1.91	Yes
T2 / 300	Momentum	0.01	82.5	79.9	0.07	1.20	No

10.6 Debugging checklist

Optimization সমস্যা হলে

- Loss scale ও label ঠিক?
- Gradient finite?
- LR range test হয়েছে?
- `zero_grad()` সঠিক স্থানে?
- Regularizer total loss-এ যোগ হয়েছে?
- Train/eval mode ঠিক?

Continual সমস্যা হলে

- Old model/parameters detached clone?
- Importance nonzero ও normalized?
- Task A before/after একই test set?
- Optimizer state boundary handling?
- Replay balance ঠিক?
- Multiple random seeds?

Implementation trap

Regularization function তৈরি করলেই regularization হয় না। Training loop-এ `total_loss = task_loss + regularizer(model)` এবং তারপর `total_loss.backward()` করতে হবে। পুরোনো parameter snapshot-এর জন্য `parameter.detach().clone()` ব্যবহার করতে হবে, নইলে reference একই tensor-এর সাথে বদলে যেতে পারে।

Chapter Summary ও Cheat Sheet

11.1 এক নজরে

Concept	Key equation / signal	One-line meaning
SGD	$\theta \leftarrow \theta - \eta g$	Current mini-batch gradient অনুসরণ
Momentum	$v \leftarrow \beta v + g; \theta \leftarrow \theta - \eta v$	History দিয়ে direction smooth ও accelerate
Adam	$\theta \leftarrow \theta - \eta \hat{m} / (\sqrt{\hat{v}} + \epsilon)$	First moment + adaptive second-moment scaling
Landscape	$L(\theta)$	Parameter space-এ loss geometry
Curvature	$H = \nabla^2 L$	Direction-wise slope change
Conflict	$g_i \cdot g_j < 0$	এক task update অন্যটির loss বাড়াতে পারে
Cosine	$(g_i \cdot g_j) / (\ g_i\ \ g_j\)$	Scale-free direction similarity
Projection	Conflicting component remove	First-order harm কমানোর gradient surgery
Replay	$L_{\text{new}} + \alpha L_{\text{old}}$	Past gradient update-এ ফিরিয়ে আনা
SI/MAS/EWC	$\Sigma I_i (\theta_i - \theta_i^{\text{old}})^2$	Important parameter movement penalize

11.2 দশটি মূল takeaway

1. Gradient steepest ascent direction; negative gradient local descent direction।
2. Mini-batch noise optimization path বদলায় এবং কখনও exploration/generalization-এ সাহায্য করে।
3. Momentum persistent direction accumulate করে, oscillating direction cancel করে।
4. Adam direction history এবং gradient scale—দুটিই estimate করে।
5. Optimizer comparison fair করতে আলাদা learning-rate tuning দরকার।
6. Loss landscape high-dimensional; 2D visualization একটি slice, পুরো সত্য নয়।
7. Negative gradient dot product first-order conflict নির্দেশ করে।
8. Interference হলো observed damage; conflict হলো local geometric signal।
9. Continual learning stability ও plasticity-এর balance খোঁজে।
10. Reliable conclusion-এর জন্য accuracy, forgetting, cosine, norms, seeds ও ablation একসাথে দরকার।

11.3 Short questions

1. SGD-এর stochasticity কোথা থেকে আসে?
2. Momentum কেন narrow valley-তে SGD-এর চেয়ে দ্রুত হতে পারে?
3. Adam-এ bias correction কেন দরকার?
4. Adaptive learning rate এবং learning-rate-free হওয়া কেন এক নয়?
5. Hessian-এর negative eigenvalue কী নির্দেশ করে?

6. Gradient cosine negative হলে কী first-order prediction করা যায়?
7. Gradient conflict ও catastrophic forgetting কি একই জিনিস?
8. Replay এবং parameter regularization কীভাবে আলাদা?

11.4 Analytical problems

1. $\theta=[1,-2]$, $g=[-0.5,1.5]$, $\eta=0.2$ হলে এক SGD step-এর পর θ' বের করুন।
2. $\beta=0.8$, $v_{t-1}=[1,-1]$, $g_t=[0.5,0.25]$ হলে v_t বের করুন।
3. $g_1=[1,2]$ ও $g_2=[-3,1]$ -এর dot product, cosine sign এবং relationship নির্ধারণ করুন।
4. Task A accuracy 89% থেকে 76%, এবং Task B accuracy 91% হলে forgetting percentage-point এবং relative drop বের করুন।
5. দুই task gradient conflict করলে g_1 -কে g_2 -এর normal plane-এ project করার formula লিখুন।

11.5 Suggested answers

1. $\theta'=[1,-2]-0.2[-0.5,1.5]=[1.1,-2.3]$ ।
2. ব্যবহৃত convention $v_t=\beta v_{t-1}+g_t$ হলে $v_t=[1.3,-0.55]$ । যদি $(1-\beta)g$ convention হয়, ফল আলাদা; equation আগে উল্লেখ করতে হবে।
3. Dot = $1(-3)+2(1)=-1$; cosine negative, তাই gradients conflicting।
4. Forgetting = 13 percentage points; relative drop = $13/89 \times 100 \approx 14.61\%$ ।
5. $g'_1=g_1-[(g_1^T g_2)/\|g_2\|^2]g_2$, সাধারণত conflict থাকলেই প্রয়োগ করা হয়।

11.6 Long-answer prompts

1. SGD, Momentum এবং Adam-এর update rule, state, memory cost, convergence behavior ও generalization trade-off তুলনা করুন।
2. Loss landscape-এর curvature ও condition number ব্যবহার করে SGD oscillation এবং momentum acceleration ব্যাখ্যা করুন।
3. Continual learning-এ gradient conflict কীভাবে forgetting ঘটতে পারে তা Taylor approximation দিয়ে দেখান এবং অন্তত তিনটি mitigation strategy তুলনা করুন।
4. Gradient cosine alone কেন insufficient diagnostic—dot product, norm, curvature এবং functional interference-এর আলোকে আলোচনা করুন।

Final perspective

Optimization এবং continual learning আলাদা বিষয় নয়। Optimizer প্রতিটি step-এ যে direction ও scale বেছে নেয়, সেটিই loss landscape-এর মধ্যে model-এর trajectory তৈরি করে। বহু task থাকলে সেই trajectory একাধিক gradient-এর negotiation। এই geometry বুঝতে পারলে SGD/Adam বেছে নেওয়া, forgetting diagnose করা এবং SI/MAS/replay/projection-এর মতো method design করা অনেক বেশি systematic হয়।